Research Summary

Bingnan Chen
Institute of Computer Science
University of Science and Technology of China

Supervisors:
Yinchi Cao (JHU)
Junchen Jiang (UChicago)
Panlong Yang (USTC)

Outline

- Research on cross-site (XSS) based on taint tracking for Chromium v8 - 2020.7 ~
- Yoda: A Video analytic pipeline Benchmark - 2020.7 ~ 2020.8
- User Activity recognition and Identification through Wireless Backscattering - 2019.9 ~ 2020.1
- C minus Compiler - 2019.10~2019.12
- Recent projects - 2020.3~2020.7

I. Cross-site (XSS) based on Taint Tracking for Chromium v8 - Motivition

- As complex JavaScript application become more widespread, DOM (Document Object Model) XSS vulnerabilities are becoming more and more common.

I. Cross-site (XSS) based on Taint Tracking for Chromium v8 - Overview

I. Cross-site (XSS) based on Taint Tracking for Chromium v8 - Idea

- Previous: Method of confirming the attack was relatively naive and not comprehensive enough. This method basically appends a value after the url.
- Generated injection URL:
  `example.url.com/path?param=test&a=b#`**`INJECT`**
- Disadvantage: Many cases are not detected.

I. Cross-site (XSS) based on Taint Tracking for Chromium v8 - Idea

- Now: Through constraint solver, trying to find the value not filtered by the sanitizer and the value which can also trigger the exploit.

- Advantage: Propose a more accurate and more effective method of confirming attacks.

I. Cross-site (XSS) based on Taint Tracking for Chromium v8 - Implementation

- (1) Find the taint flow, extract all the details of the flow (string operation), and put them as constraints into the constraint solver to find more exploitable flows.
- Example:

```
1:[[444, ['OnNewSubStringCopy', 'to', '8|www', 'from', '8|www.tmz.com']], [12399,
['OnJoinManyStrings', 'to', '8|www.tmz.com', 'from', '8|www']], [13104, ['NewConsString',
'to', '8|https://www.tmz.com', 'from', '8|www.tmz.com']], [13105, ['NewConsString', 'to',
'18|https://www.tmz.com/', 'from', '8|https://www.tmz.com']], [22,
['Sink_function:HTMLAnchorElement.parseAttribute', 'https://www.tmz.com/']]]
```

I. Cross-site (XSS) based on Taint Tracking for Chromium v8 - Implementation

- (2) Make the sink contains these attack vectors, and then leverage the constraint solver to solve the corresponding string value in the source.
- Example:
- document.write('some html' + str);
- str = '<script>alert(1);</script>'

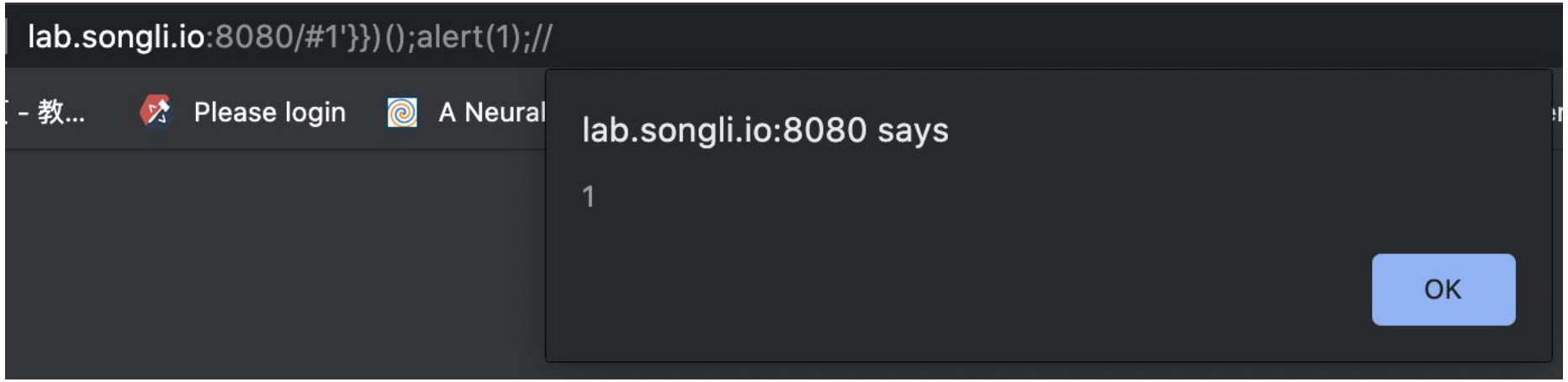
I. Cross-site (XSS) based on Taint Tracking for Chromium v8 - Implementation

- (3) Assign this value to the source and run Chromium.
- Example:
- url = "http://lab_songli_io:8080/#1'}})();alert(1);//"
- head = "(function A(){ myjson={'url':'";
- tail = "'};console.log(myjson)})();";
- content = head.concat(url, tail);

```
(function A(){ myjson={'url':'http://lab_songli_io:8080/#1'}})();alert(1);//'};console.log(myjson)})();
```

I. Cross-site (XSS) based on Taint Tracking for Chromium v8 - Implementation

- (4) If the web page displays a message about an attack vector, it means that absorbing an exploit successfully.



I. Cross-site (XSS) based on Taint Tracking for Chromium v8 - Implementation

- (5) In this way, I can identify the value of the source that cannot be filtered out by the sanitizer and can also trigger the exploit.

- Previous example: A successful one!

II. Yoda: A Video analytic pipeline Benchmark - Background

- Video analytics pipelines (VAPs): Seek to reduce network/compute costs without hurting accuracy.

- Primitives: Many VAPs share similar techniques/ideas, so it's efficient to modularize a VAP into cost-saving strategies. These strategies fall into the following three primitives.

II. Yoda: A Video analytic pipeline Benchmark - Background

VAP	*Temporal pruning*	*Spatial pruning*	*Model pruning*
VideoStorm[71]	✔ (uniform sampling)	✔(quality downsize)	✔ (model selection)
NoScope[48]	✔ (diff-triggered)		✔(specialization)
AWStream[70]	✔ (uniform sampling)	✔(quality downsize)	
Glimpse[26]	✔ (diff-triggered)		✔(fixed tiny model)
Vigil[72]	✔ (diff-triggered)	✔ (region cropping)	✔ (fixed tiny model)
Chameleon[46]	✔ (uniform sampling)	✔(quality downsize)	✔(model selection)
VideoEdge[43]	✔(uniform sampling)	✔(quality downsize)	
DDS[60]		✔ (region cropping)	
EAAR[53]	✔ (diff-triggered)	✔ (region cropping)	
WEG[63]			✔(specialization)

Table: Mapping each VAP to the subset used primitives

II. Yoda: A Video analytic pipeline Benchmark - Implementation

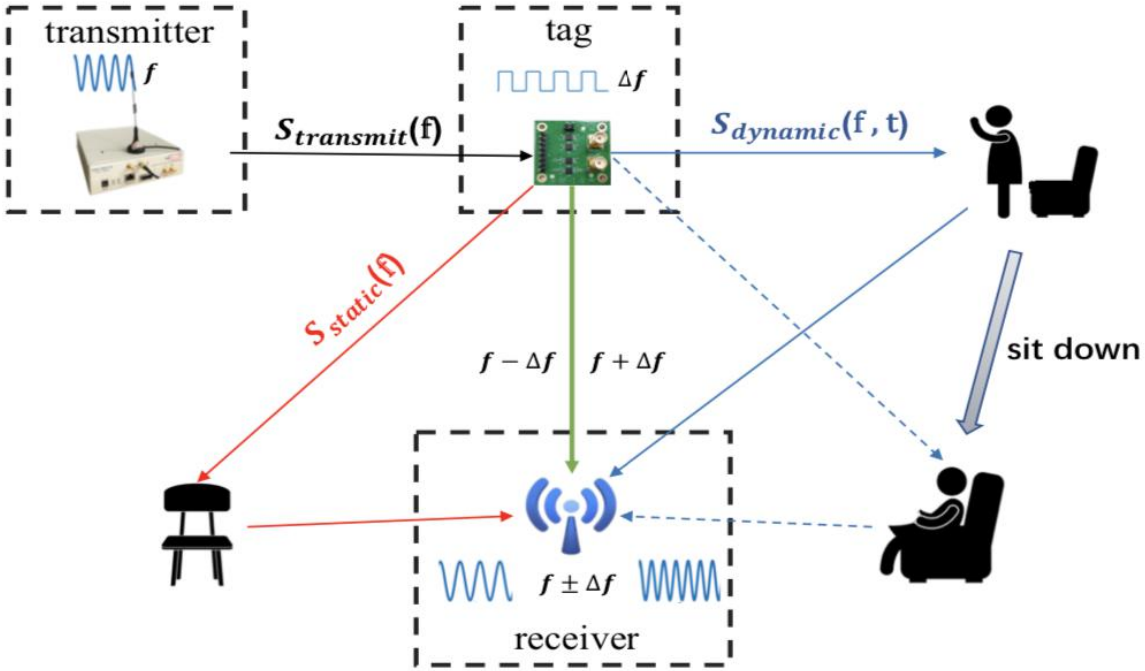
- Implement an interface for videostorm, a pipeline proposed in NSDI.

VAP	*Temporal pruning*	*Spatial pruning*	*Model pruning*
VideoStorm[71]	✔ (uniform sampling)	✔(quality downsize)	✔ (model selection)

- Implement an interface for awstream, a pipeline proposed in sigcomm.

VAP	*Temporal pruning*	*Spatial pruning*	*Model pruning*
AWStream[70]	✔ (uniform sampling)	✔(quality downsize)	

III.User Activity recognition and Identification through Wireless Backscattering --Backgrounds

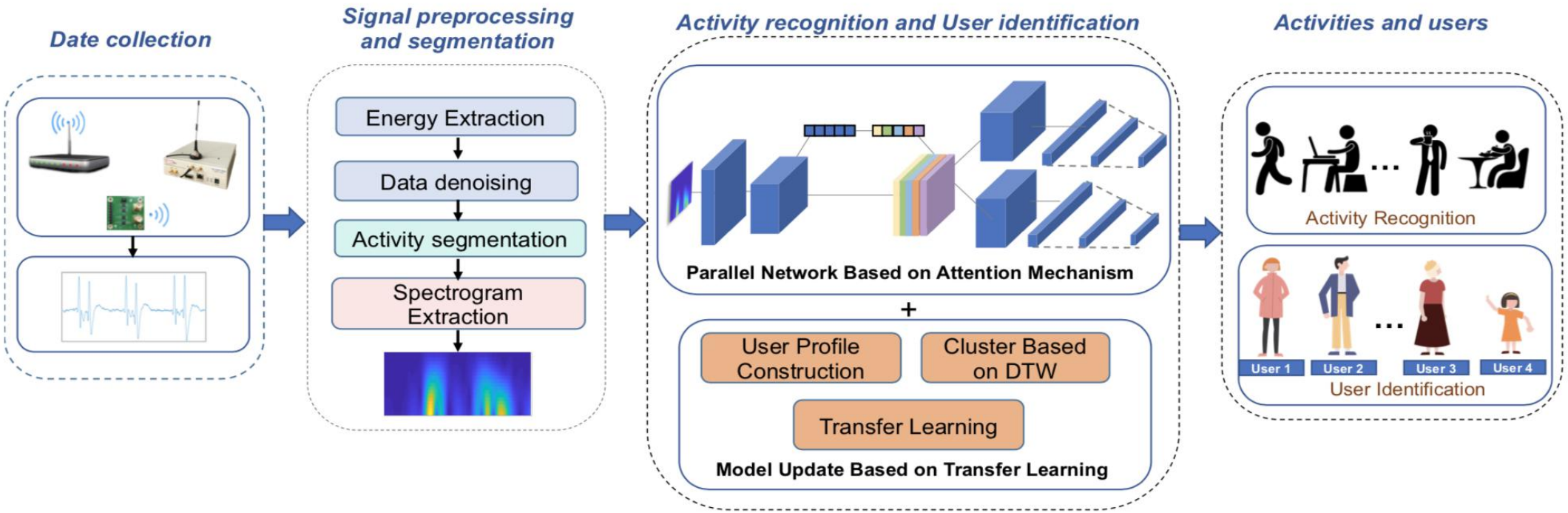


(a) Signal Propagation Path

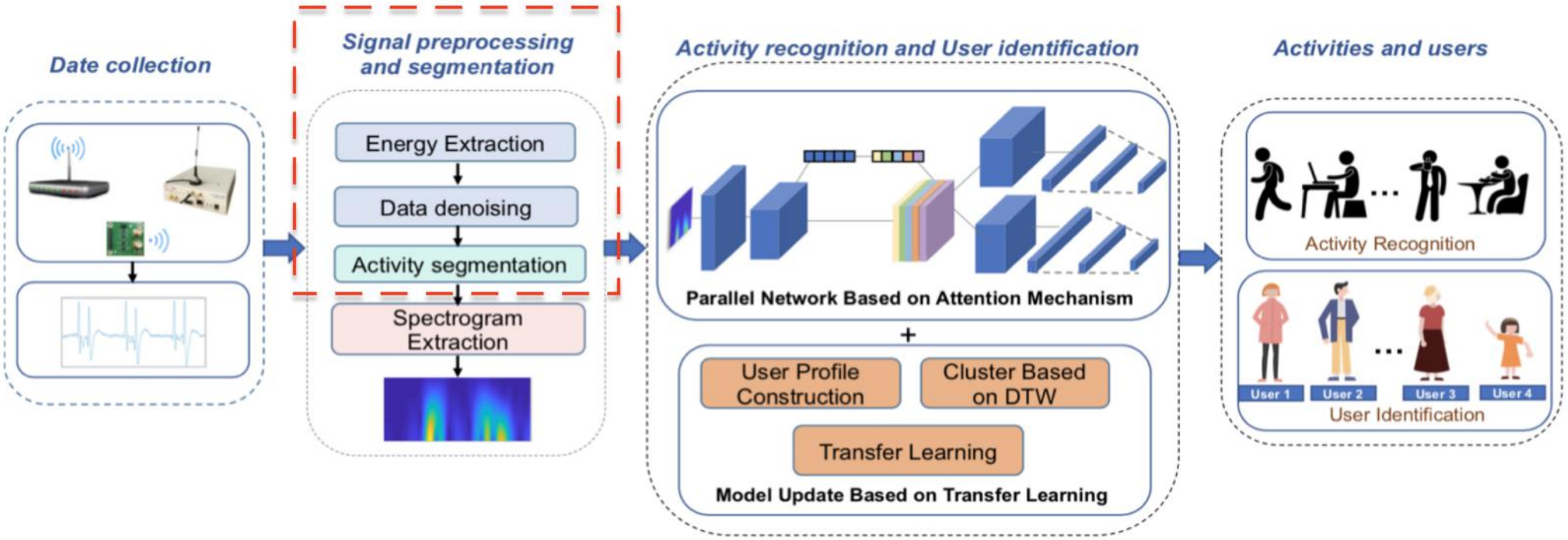
- In this system, there is one transmitter, multiple receivers, and multiple battery-free backscattering tags to reflect the signal transmitted by the transmitter.
- Srx(f,t) = Sstatic(f)+Sdynamic(f,t)
- Sdynamic(f,t)= $A(f,t)e^{\frac{-j2\pi d(t)}{\lambda}}$

Reference: Manjiang Yi, "Back-Guard: Parallel Attention Model based User Activity Recognition and Identification through Wireless Backscattering "

III.User Activity recognition and Identification through Wireless Backscattering -- Overview

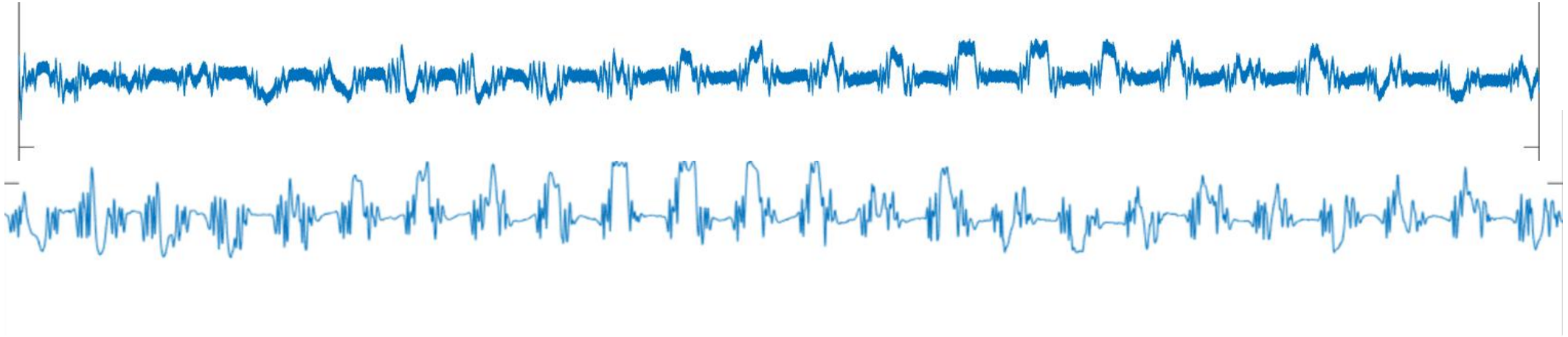


III.User Activity recognition and Identification through Wireless Backscattering



III.User Activity recognition and Identification through Wireless Backscattering --Signal Processing

- Because of the original backscattering signals extracted from the receiver are inherently noisy, I specify filters with fixed window length determined heuristically to preprocess the data.



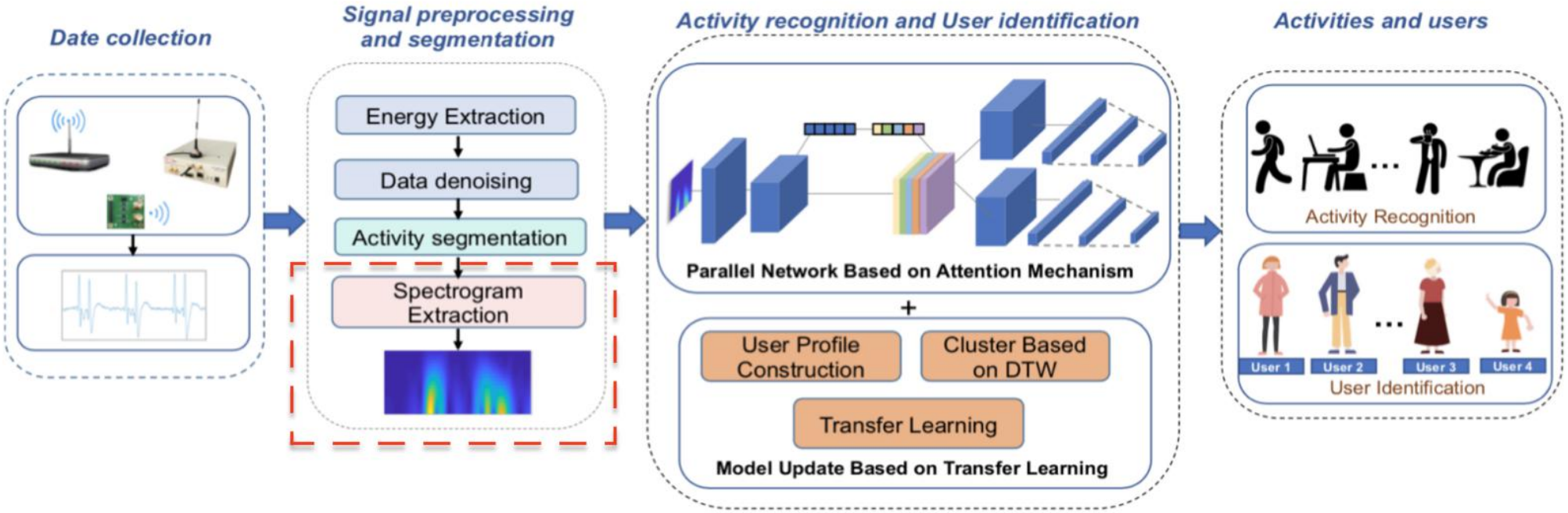
(Above are the original data and processed data.)

III.User Activity recognition and Identification through Wireless Backscattering -- Adaptive Activity Detection and Segmentation

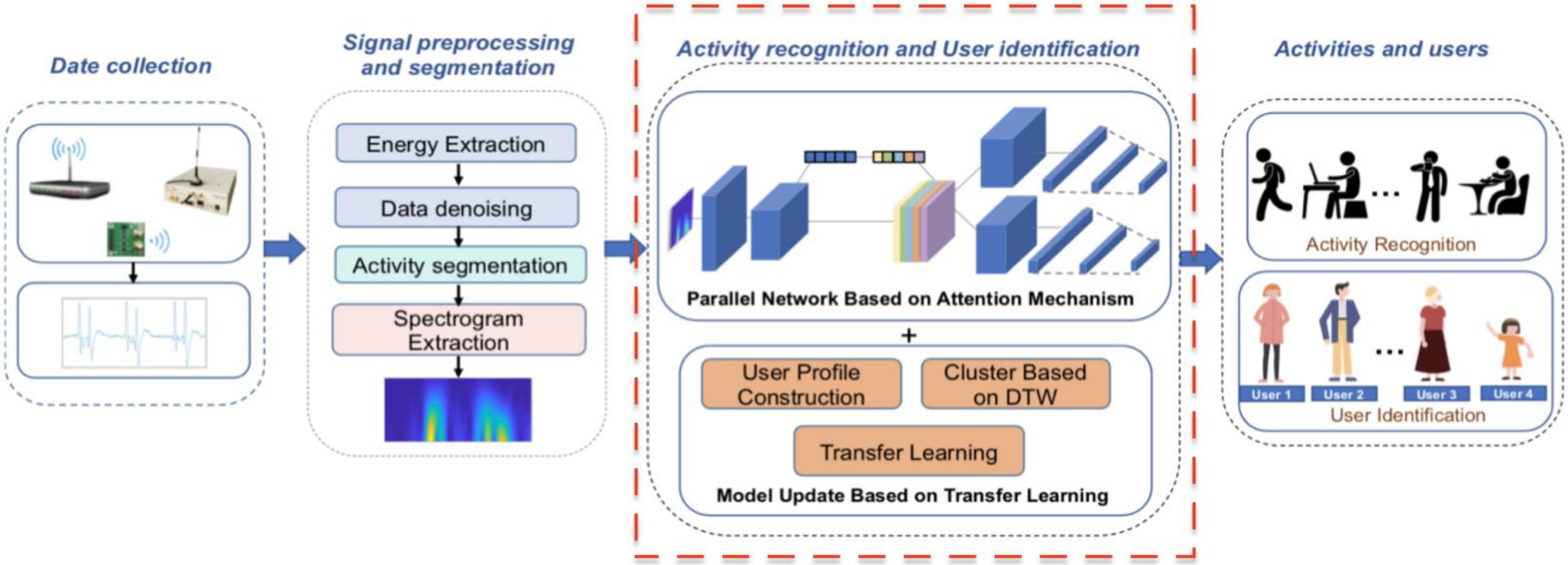
- Preliminary Activity Detection: Use STE(short time energy) and mvar(moving variance) to segment the signal.
- STE: $STE(t) = \sum_{n=1}^{N} v(t+n)^2$
- Adaptive Accurate Segmentation: Move the sliding pointer i from left to right to traverse the signal. Check whether the significance P(Tmax) exceeds a selected threshold P0. If so, the signal is cut at the point i into two subsections. otherwise the signal remains undivided.
- SD(i): $s_D(i) = \left(\frac{(N_1-1)*s_1(i)^2 + (N_2-1)*s_2(i)^2}{N_1+N_2-2}\right)^{\frac{1}{2}} * \left(\frac{1}{N_1} + \frac{1}{N_2}\right)$
- T(i): $T(i) = \left|\frac{\mu_1(i) - \mu_2(i)}{s_D(i)}\right|$
- P(Tmax): $P(max(T(i))) \leftarrow Prob(T \leq max(T(i)))$

III.User Activity recognition and Identification through Wireless Backscattering -- Deep learning Based Motion Recognition and User Identification

- Generate spectrograms: Apply Short Time Fourier Transform(STFT) to obtain time-frequency feature for back scattering signal.



III.User Activity recognition and Identification through Wireless Backscattering -- Deep learning Based Motion Recognition and User Identification



III.User Activity recognition and Identification through Wireless Backscattering -- Deep learning Based Motion Recognition and User Identification

- Feature extraction module: Use convolutional layers to extract deep features of signals for robust activity recognition and user identification.

- Attention module: Follow the SEnet to explicitly model the interdependencies between the channels of the convolutional features.

- Parallel classification module: Attention module can obtain high-level features of physiological and behavioral characteristics. Feed these features into fully connected layers to get the results of classification. The extracted features are weight-shared for two FCN layers. (user activity recognition and user identification)

IV.C minus compiler

- Lexical Analyzer: Used flex to build a lexical analyzer, doing some analyses and outputting five attributes tuple for syntax analyzer. According to the different tokens, did different actions.
- Syntax Analyzer: Used bison to build a syntax analyzer,depending on the input tokens to do certain actions and producing a syntax tree.
- Intermediate code generation: Used LLVM C++ interface to generate LLVM IR, exchanging syntax tree(generated in previous steps) to C++ syntax tree with the help of visitor mode.

V. Recent projects

- RISC_V_32I (finished): Implemented RISC_V_32I cpu, group-connected and fully-connected cache(fifo, lru), branch prediction part(btb, bht). -- language: verilog & system verilog
- Bank Management System (finished): Use vue to complete the frontend and flask to complete the backend. The main functions are to add, delete, modify, check and visualize business statistics. -- language: python & html & css & javascript
- Online Interview System (finished): Use element-ui(desktop component library of vue 2.0) and django to realize. My work is mainly focused on database design, frontend and backend data interactions and backend api design. -- language: python & html & css & javascript
- Elementary machine learning algorithms (finished): SVM, KNN, PCA, Kmeans, Logistic regression. -- language: python